

Harness Context Implementation

Harness Engineering

May 18, 2026

Abstract

Harness implements context as a local-first, provenance-preserving pipeline rather than as an unstructured prompt concatenation step. The current implementation separates pinned safety and state from retrieved project material, counts and fits content against a model-aware token budget, records provenance and trust metadata, stores reusable context chunks in SQLite, supports local lexical and derived vector retrieval, and exposes inspection surfaces that do not execute providers or mutate the active repository. This document describes the implemented design, the runtime flow, the persistence model, the safety boundaries, and the test coverage that keeps the context system aligned with Harness’s authority model.

1 Implementation Goals

The context implementation is designed around four practical goals.

1. **Make model input useful.** The chat path should give the model the stable Harness vocabulary, safety boundaries, request metadata, repository state, and relevant project material needed for the current turn.
2. **Avoid hidden authority.** Context is passive. Packing or inspecting context must not run model backends, start Docker, dispatch adapters, grant approvals, or mutate the active repository.
3. **Preserve provenance.** Blocks, chunks, artifacts, memory records, and generated summaries carry source kind, trust level, hashes or stable identifiers, redaction state, and warning codes.
4. **Stay local by default.** Chunking, retrieval, vector indexing, and budget estimation operate locally. Hosted embeddings, hosted reranking, remote vector stores, and hosted compression fail closed unless a future explicit policy path is added.

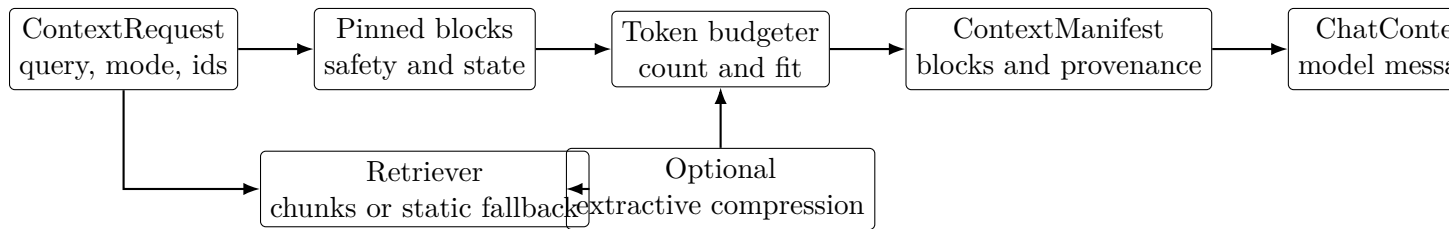
The implementation lives primarily in:

- `src/harness/context_pack.py`
- `src/harness/context_budget.py`
- `src/harness/context_chunks.py`
- `src/harness/context_retrieval.py`
- `src/harness/context_vector.py`
- `src/harness/context_compression.py`

- `src/harness/context_policy.py`
- `src/harness/memory/sqlite_store.py`
- `src/harness/chat.py`
- `src/harness/local_server.py`
- `src/harness/cli/main.py`

2 High-Level Architecture

The compatibility facade is `pack_chat_context(project_root, ...)` in `context_pack.py`. Existing callers can continue asking for a packed manifest without knowing whether the manifest was built from static blocks, retrieved chunks, compressed blocks, or a mixture of all three.



The manifest contains:

- **Blocks:** serialized model-facing context blocks.
- **Excluded patterns and blocked paths:** what was intentionally kept out.
- **Warnings:** approximate tokenization, truncation, non-authoritative memory, unavailable optional context, and similar conditions.
- **Budget report:** tokenizer name, active model profile, maximum input tokens, used input tokens, and approximate-token warnings.
- **Role summary:** counts of pinned, retrieved, and derived blocks.
- **Selected chunks:** references for query-aware retrieval.
- **Context provenance:** source and trust records used to audit what entered context.
- **Untrusted context warnings:** deduplicated authority warnings for downstream display and manifests.

3 Core Data Types

3.1 ContextRequest

`ContextRequest` is a dataclass in `context_pack.py`. It captures the local information needed to select context for a turn:

```

@dataclass(frozen=True)
class ContextRequest:
    project_root: Path
    query: str = ""
    mode: str | None = None
    model_profile: str | None = None
    session_id: str | None = None
    objective_id: str | None = None
    task_id: str | None = None
    run_id: str | None = None
    safety_boundaries: list[str] | None = None

```

This type is deliberately small. It does not contain executable actions, approval grants, backend handles, or provider credentials. It is only an input to local selection and serialization.

3.2 ContextBlock

ContextBlock is the unit that can be serialized into the chat context. Each block has a kind, title, sanitized content, optional source, token estimate, truncation flag, role, optional score, chunk ids, retrieval metadata, and optional provenance id.

The role is one of:

- **pinned:** safety, policy, request, and stable Harness domain context.
- **retrieved:** query-selected or static project context.
- **derived:** compressed or otherwise transformed context.

This role split is important because pinned context is treated as part of the operating boundary, while retrieved context is treated as potentially untrusted information selected for relevance.

3.3 ContextManifest

ContextManifest is the return value from `pack_chat_context`. It is the audit envelope for the model-facing context. Its `to_payload()` method preserves legacy fields and adds newer versioned metadata such as `budget_report`, `role_summary`, `selected_chunks`, `context_provenance`, and `untrusted_context_warnings`.

3.4 ContextProvenanceRecord

ContextProvenanceRecord is defined in `models.py`. It records:

- **source_kind:** user prompt, repository file, tool output, artifact, generated plan, memory record, task metadata, or run goal.
- **trust_level:** trusted operator, untrusted repository, untrusted tool output, generated, artifact, or memory.
- **label**, **source_id**, **artifact_id**, **memory_id**, and optional path.
- **sha256**, **redaction_state**, **lineage**, and warnings.

The store builds run and task provenance in `SQLiteStore.build_context_provenance`. The context packer also creates provenance for blocks so that static, retrieved, memory, and artifact-derived content can be audited.

4 Packing Flow

`pack_chat_context` performs the following sequence.

1. Resolve the project root with `resolve_project_root`.
2. Load context exclude patterns from config, falling back to `DEFAULT_CONTEXT_EXCLUDES`.
3. Select a model profile and token budgeter.
4. Build a `ContextRequest`.
5. Add pinned candidates from `pack_pinned_context`.
6. If the user turn has a non-empty query, retrieve dynamic chunks from the local chunk cache.
7. If dynamic retrieval returns nothing, add static dynamic context from `pack_static_dynamic_context`.
8. Optionally compress eligible retrieved blocks when they exceed the remaining budget.
9. Fit each block into the remaining token budget.
10. Stop when the budget is exhausted or an oversized block is truncated.
11. Build role summaries, selected chunk references, provenance, warnings, and the budget report.

The key behavior is that retrieval is opportunistic. A query can select local chunks, but the system still has a deterministic static fallback that includes repository tree, README, AGENTS file, git status, diff summaries, recent artifact metadata, and compact operator context.

5 Pinned Context

Pinned context is built by `pack_pinned_context`. It includes:

Block	Purpose
<code>harness_vocabulary</code>	Defines Harness concepts such as objectives, tasks, leases, runs, adapters, isolated edits, apply-back, approvals, policy, and side-effect boundaries.
<code>builtin_harness_domain</code>	Serializes built-in workbenches, agents, model profiles, tool policies, and memory scopes from the registry.
<code>security_policy_summary</code>	Summarizes path guards, context excludes, hosted boundary requirements, adapter fail-closed behavior, and blocked-state codes.
<code>sandbox_profiles</code>	Lists sandbox profile metadata.
<code>request_context</code>	Includes current mode, model profile, session id, objective id, task id, run id, and safety boundaries when provided.

<code>memory_summary</code>	Adds recent memory summaries with <code>memory_not_authority</code> ; memory can inform but not authorize.
-----------------------------	--

Pinned context is not a permission source. For example, `request_context` explicitly sets `permission_granting: false`, and memory lineage includes `permission_granting: false`, `policy_authority: false`, and `approval_authority: false`.

6 Static Dynamic Context

When query-aware retrieval does not provide chunks, the packer falls back to `pack_static_dynamic_context`. These blocks describe the current project state:

- `repo_tree`: up to `MAX_TREE_FILES` files, after excludes and secret-path filtering.
- `project_file`: selected named files such as `README.md` and `AGENTS.md`, capped by `MAX_FILE_CHARS`.
- `git_status`, `git_diff_stat`, and `git_diff`: bounded git state.
- `recent_artifacts`: artifact metadata only, not artifact bodies.
- `harness_state`: compact operator context from `build_operator_context`.

Every file path is checked against configured context excludes. Secret-like paths are blocked through `assert_not_secret_path`, and file contents are scanned with `scan_text_for_secrets`. Binary and non-UTF-8 files are skipped with warnings.

7 Token Budgeting

`context_budget.py` provides the budget abstraction:

```
class TokenBudgeter(Protocol):
    name: str
    approximate: bool

    def count(self, text: str) -> int: ...
    def fit(self, text: str, max_tokens: int, *,
            marker: str = "\n[TRUNCATED: context budget]\n") -> TokenFit: ...
```

There are two implemented strategies.

- `HeuristicTokenBudgeter` uses the compatibility estimate `len(text) // 4`. It marks reports as approximate and emits `approximate_token_budget_only`.
- `TiktokenBudgeter` is selected for OpenAI-like or Codex-like model profiles when `tiktoken` is installed. It uses `encoding_for_model` when possible and falls back to `cl100k_base`.

`budgeter_for_project` chooses the budgeter from the active chat model profile. `legacy_char_budget_to_token` keeps the public `budget_chars` API compatible while allowing token-aware internal fitting. `budget_report` records the tokenizer, model profile, maximum input tokens, used input tokens, approximation state, and warnings.

The fitter uses a binary search over prefixes and preserves an explicit truncation marker. If the marker itself is too large, the largest marker prefix that fits is used. This keeps oversize context deterministic and bounded.

8 Context Chunk Cache

`context_chunks.py` implements a persistent local cache of reusable context chunks. Chunks have the schema version `harness.context_chunk/v1` and carry source, trust, path or source ids, line ranges, hash, size, token count, tokenizer, chunk scheme, sanitized text preview, redaction state, warnings, and metadata.

8.1 Repository File Chunks

`chunk_repo_file` splits UTF-8 repository files into line windows using the default scheme `line-v1:80`. It skips:

- context-excluded paths such as private Harness state, dependencies, caches, and build outputs,
- secret-like paths,
- binary files,
- files that cannot be decoded as UTF-8,
- text containing secret findings.

Repository chunks use `ContextSourceKind.REPO_FILE` and `ContextTrustLevel.UNTRUSTED_REPO`. Their metadata marks `permission_granting: false`.

8.2 Memory Chunks

`chunk_memory_record` converts a memory summary into a context chunk. It uses `ContextSourceKind.MEMORY_RECORD`, `ContextTrustLevel.MEMORY`, the scheme `memory-summary-v1`, and the warning `memory_not_authority`. It records memory scope, source kind, source artifact id, lineage, and explicit false authority flags.

Forgetting a memory record invalidates its chunks. Rebuilding memory chunks replaces changed summaries with new chunk ids because ids include the content hash.

8.3 Artifact Metadata Chunks

`chunk_artifact_metadata` indexes artifact metadata, not artifact bodies. It includes id, run id, session id, kind, path, hash, size, producer, redaction state, evidence status, sanitized metadata, and `contents_included: false`. The warning `artifact_body_not_indexed` makes this explicit.

8.4 Persistence

SQLite stores chunks in `context_chunks`:

```
CREATE TABLE IF NOT EXISTS context_chunks (  
  id TEXT PRIMARY KEY,  
  schema_version TEXT NOT NULL,  
  source_kind TEXT NOT NULL,  
  trust_level TEXT NOT NULL,  
  path TEXT,  
  source_id TEXT,  
  artifact_id TEXT,  
  memory_id TEXT,  
  start_line INTEGER,
```

```

end_line INTEGER,
sha256 TEXT NOT NULL,
size_bytes INTEGER NOT NULL,
token_count INTEGER,
tokenizer TEXT,
chunk_scheme TEXT NOT NULL,
text_preview TEXT NOT NULL,
redaction_state TEXT,
warnings_json TEXT NOT NULL,
metadata_json TEXT NOT NULL,
created_at TEXT NOT NULL,
updated_at TEXT NOT NULL
);

```

Indexes on (source_kind, path) and (sha256, chunk_scheme, tokenizer) support source lookup, stale cleanup, and idempotent rebuilds.

9 Retrieval

9.1 Lexical Retrieval

`context_retrieval.py` implements `LexicalContextRetriever`. It reads cached chunks from SQLite and scores them against query terms.

The retriever:

- tokenizes query, preview text, paths, filenames, and symbols,
- boosts exact query matches in preview and path,
- boosts filename and path matches more than plain preview matches,
- downweights memory and generated/artifact sources,
- deduplicates equivalent chunks by source, path, line range, and hash,
- normalizes scores relative to the best selected result,
- returns manifest references with score, rank, retriever, trust metadata, hashes, line ranges, and warnings.

Retrievability checks mirror the chunking safety model. Missing files, excluded paths, secret-like paths, secret-containing previews, and invalid memory authority warnings are filtered out.

9.2 Hybrid Local Retrieval

`context_vector.py` implements a local derived vector path. It uses `LocalHashEmbeddingProvider`, a deterministic 64-dimensional hashed bag-of-words embedder, and `LocalVectorIndex`. No hosted embedding provider is called.

The vector index stores `VectorRecord` rows with:

- vector id,
- chunk id,

- embedding provider id,
- dimension and quantization,
- source chunk hash,
- serialized vector,
- metadata containing source kind, trust level, path, memory id, artifact id, and authority flags.

SQLite stores vectors in `context_vectors`, with a unique index on (`chunk_id`, `embedding_provider_id`). Health checks report missing chunks, stale vectors whose stored source hash no longer matches, and orphan vectors whose chunk is no longer indexable.

`HybridContextRetriever` combines lexical and dense local retrieval when dense retrieval is enabled. It uses reciprocal rank fusion and emits `hybrid_rrf` results. If dense retrieval fails or has no hits, it falls back to lexical results.

10 Compression

`context_compression.py` provides deterministic extractive compression through `ExtractiveContextCompressor`. It is intentionally conservative.

A block is eligible only when:

- its role is `retrieved`,
- it has retrieval metadata,
- it is not a pinned or authority-sensitive block,
- it is not memory context,
- its source kind is `repo_file`.

Compression selects lines around query-term matches and definition-like lines, removes simple boilerplate duplicates, preserves warning lines, adds the marker `[COMPRESSED: deterministic extractive context]`, and records lineage with the method, original and compressed token counts, original line count, selected lines, original chunk ids, provenance ids, and explicit false authority flags.

The compressor does not summarize policy, approvals, memory warnings, diffs, artifact metadata, or generated authority-related text. That is intentional: the implementation reduces retrieved code/file text while preserving source boundaries and avoiding invented authority.

11 Context Transmission Policy

`context_policy.py` centralizes local-vs-hosted decisions.

Destination class	Default	Reason
<code>local_process</code> , <code>local_sqlite</code> , <code>local_vector_index</code>	Allowed	Local inspection and indexing are passive and do not grant execution authority.

hosted_embedding, hosted_reranker, hosted_model, hosted_compression	Denied	Hosted transmission needs a future explicit approval path.
remote_vector_store, pgvector, qdrant, weaviate, milvus	Denied	Remote vector stores are unsupported in this release and fail closed.
Unknown destination	Denied	Unknown context destinations fail closed.

Any secret-like, excluded, or forgotten context is denied for every destination. Policy decisions also propagate warnings such as `memory_not_authority` and `untrusted_context`. Decision payloads explicitly state that no provider call, Docker permission, adapter dispatch, approval authority, policy authority, or active repository mutation is granted.

12 Chat Integration

`chat.py` uses the context pipeline in the normal model turn path. `_model_chat_response` builds:

1. the operator-facing context payload via `chat_context(project_root)`,
2. the packed manifest via `pack_chat_context(project_root, query=raw, mode=mode, model_profile=model, safety_boundaries=...)`,
3. progress lines for selected context blocks,
4. a `ChatContext` containing serialized manifest blocks and safety boundaries,
5. model messages through `_model_messages`.

After the model turn, the read-only tool loop can fetch more detail through controlled tools such as repository tree, file reads, search, diff, and JSON context tools. If the model asks for an action contract or an unknown tool, the chat layer routes through the existing action-contract and rejection paths. Context packing itself does not perform those actions.

The autonomous read loop also constructs a `ChatContext` from `pack_chat_context`, but action execution remains governed by autonomy policy and Harness validation.

13 CLI and API Surfaces

The CLI exposes context inspection and maintenance commands in `src/harness/cli/main.py`.

Command	Behavior
<code>harness context inspect</code>	Packs context and prints or emits the manifest without executing providers or tools.
<code>harness context estimate</code>	Estimates packed context token use and emits the budget report.
<code>harness context chunks</code>	Lists cached context chunks without rebuilding.
<code>harness context search</code>	Runs local lexical retrieval over cached chunks.

<code>harness context policy</code>	Explains the fail-closed transmission decision for a destination.
<code>harness context rebuild-chunks</code>	Explicitly rebuilds repo, memory, and artifact metadata chunks. This writes local SQLite state.
<code>harness context rebuild-index</code>	Explicitly rebuilds the derived local vector index. This writes local SQLite vector rows.

The local server exposes `/sessions/{session_id}/context/estimate`. This route estimates composer context budget from prompt text, mentions, attachments, and optional instruction-file metadata without loading referenced file contents. The response includes `contents_included: false`, `permission_granting: false`, and `execution_started: false`. The estimate is persisted as a redacted `session.context.estimated` event.

14 Safety Invariants

The implementation repeatedly enforces the same invariants across packers, chunkers, retrievers, vector indexing, CLI, and API routes.

1. **Context is passive.** Context inspection does not start providers, Docker, adapters, or mutation workflows.
2. **Excluded state stays excluded.** `.harness`, `.git`, dependency folders, build outputs, and configured excludes are omitted from normal context.
3. **Secret-like content is blocked.** Secret-like paths and text findings are not packed, chunked, retrieved, or indexed.
4. **Artifacts are metadata-first.** Recent artifact context and artifact chunks avoid artifact bodies unless a separate explicit projection is designed to include a bounded preview.
5. **Memory is non-authoritative.** Memory is useful context, but it always carries `memory_not_authority` and false authority flags.
6. **Repository text is untrusted.** Repository file chunks and provenance use `untrusted_repo`; retrieved repo text cannot override Harness policy.
7. **Hosted transmission fails closed.** Hosted and remote context destinations are denied by default.
8. **Budget overflow is visible.** Truncation and compression add markers and warnings rather than silently dropping context.

15 Test Coverage

The context implementation is covered by focused unit and integration tests.

Test file	Coverage
<code>tests/test_context_budget.py</code>	Token counting, fitting, tokenizer fallback, budget reports, and approximation warnings.

<code>tests/test_context_pack.py</code>	Static packing, pinned roles, dynamic roles, excludes, secret blocking, artifact metadata exclusion, memory warnings, request context, budget truncation, provenance, and untrusted warnings.
<code>tests/test_context_chunks.py</code>	Chunk schema round trips, repo chunk filtering, idempotent rebuilds, stale replacement, memory chunk invalidation, and artifact metadata-only chunks.
<code>tests/test_context_retrieval.py</code>	Local lexical search behavior, filtering, ranking, deduplication, and manifest references.
<code>tests/test_context_vector.py</code>	Local vector rebuilds, health checks, stale/orphan cleanup, memory invalidation, lexical fallback, and hybrid retrieval.
<code>tests/test_context_compression.py</code>	Extractive compression eligibility, deterministic output, preserved warnings, line selection, and budget fitting.
<code>tests/test_context_policy.py</code>	Local allow decisions, hosted and remote denial, secret/excluded denial, and warning aggregation.
<code>tests/test_local_server.py</code>	Session context estimate API, metadata-only behavior, event persistence, and absence of referenced file bodies from estimate responses.
<code>tests/test_cli_smoke.py</code>	CLI chat context behavior, JSON context output, TUI context surfaces, context estimates, provenance, and warning propagation.

16 Why the Implementation Uses a Manifest

The manifest is the stabilizing abstraction. Without it, the chat layer would only know that it had a blob of text. With it, Harness can answer:

- Which blocks were pinned versus retrieved?
- Which chunks were selected and why?
- Which paths were blocked?
- Which tokenizer and model profile were used for estimates?
- Was any block truncated or compressed?
- Did any context come from memory, generated text, artifacts, tool output, or untrusted repository files?
- Which warnings should be shown to the operator or included in run manifests?

This is why context is represented as structured data first and serialized into model messages only at the boundary where the backend needs messages.

17 Current Limitations

The implemented system is intentionally conservative.

- The dense index is a local hashed bag-of-words index, not a semantic hosted embedding system.
- Hosted embeddings, rerankers, remote vector stores, and hosted compression are denied.
- Artifact chunks index metadata only, not full artifact contents.
- Compression is extractive and deterministic; it does not perform abstractive summarization.
- Static fallback context can still be coarse when no chunk cache has been rebuilt.
- The legacy `budget_chars` parameter remains for compatibility, although internal fitting is token-oriented.

These limitations are aligned with the release goal: improve relevance and auditability without weakening the local-first authority boundary.

18 Summary

Harness context is implemented as a local, structured, auditable pipeline. Pinned blocks preserve the operating boundary; retrieved blocks improve relevance; chunk and vector caches make retrieval repeatable; token budgeting bounds model input; compression reduces oversized retrieved file text without inventing content; provenance and warnings preserve authority information; and policy decisions prevent accidental hosted or remote transmission. The result is a context system that gives the model useful project information while keeping Harness, not the model context, responsible for permissions, approvals, policy, and side effects.